

0.0. Le programme qui ne fait rien

Le squelette de tout programme : `int main()`, et la boucle `while`.

1. Ce qu'il faut comprendre

`int main()` est le point de départ. Quand la console s'allume, c'est là qu'elle arrive, et nulle part ailleurs. Tout ce que fait le programme s'écrit entre ses accolades `{ }`.

`while (true)` veut dire « tant que vrai » : ce qui est entre ses accolades recommence **sans jamais s'arrêter**. C'est la **boucle de jeu**. Un jeu ne se termine pas tout seul : il attend le joueur.

`image()` attend la prochaine image de l'écran. Il y en a **60 par seconde** : la boucle fait donc un tour tous les soixantièmes de seconde.

Ce programme ne montre rien, mais il tourne. Tous les suivants partent de lui : on ajoutera des choses **avant** la boucle (ce qui se fait une fois) et **dedans** (ce qui se fait à chaque image).

2. Le programme, morceau par morceau

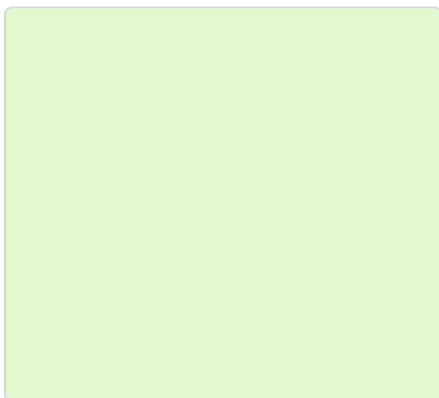
Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1273 octets** de cartouche.

1. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1

4. Ce qu'on doit voir

Rien : l'écran reste vide, mais la console tourne.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ l'écran est allumé
- ✓ et rien n'y est écrit

Ensuite : **0.1. Afficher la lettre A** — niveau 0 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++